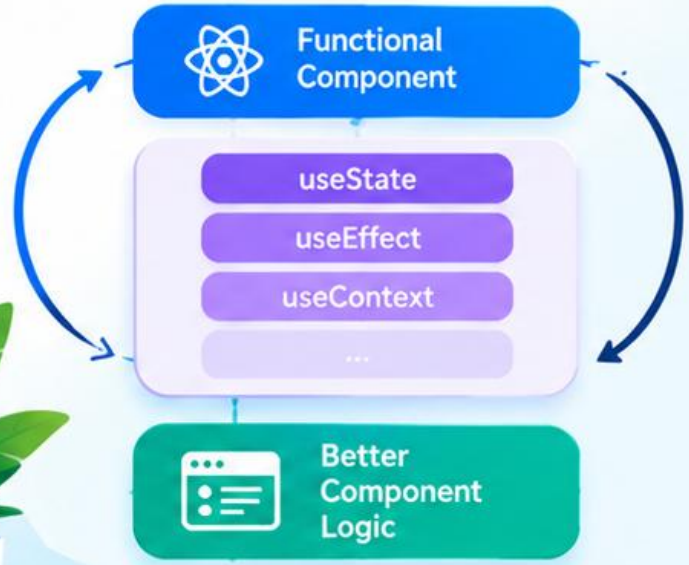
React Hooks

Use State, Life Cycle and More in Functional Components

Hooks let you use React features without writing class components.





Simple. Powerful. Flexible.

UserProfile.jsx

```jsx
import React, { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';

function UserProfile() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const { id } = useParams();

  useEffect(() => {
    const fetchUser = async () => {
      const res = await fetch('/api/users/${id}');
      const data = await res.json();
      setUser(data);
      setLoading(false);
    };
    fetchUser();
  }, [id]);

  if (loading) return <p>Loading...</p>;
  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
    </div>
  );
}
```

Build Smarter Components with React Hooks

Key React Hooks

useState
Adds state to functional components.
Manage component data

useEffect
Handles side effects like API calls, subscriptions, etc.
Run code after render

useContext
Accesses context values without prop drilling.
Share global data

useMemo
Memoizes expensive calculations.
Improve performance

useCallback
Returns a memoized callback function.
Prevent unnecessary re-renders

useRef
Accesses DOM elements or stores mutable values.
Persist values across renders

Hooks make React more powerful!

React Hooks

❖ **React Hooks**

➢ **What are React Hooks?**

- React Hooks are special functions introduced in React 16.8 that allow functional components to use state, lifecycle features, and other React functionalities without writing class components.

❖ **Key Points**

✓ Introduced in React 16.8.

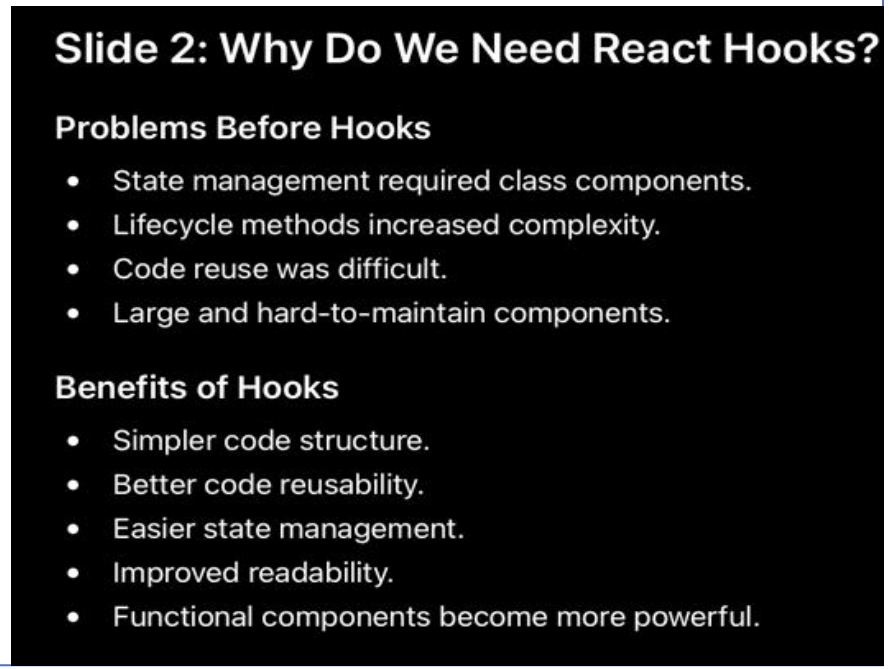
✓ Used inside functional components.

✓ Allow state management.

✓ Enable lifecycle operations.

✓ Make code cleaner and reusable.

✓ Reduce the need for class components.



Cont. ...

1. What are React Hooks?

1.1 What does React Hooks mean?

React Hooks are special functions that allow functional components to use React features such as state, lifecycle-related behavior, context, and other capabilities.

Before Hooks were introduced in React 16.8, developers primarily used class components to manage state and lifecycle methods.

With Hooks, we can implement these features using functional components.

For example, imagine you are developing a Student Management System.

You need to:

- Store student information.
- Update student records.
- Display loading messages.
- Fetch data from a backend API.
- Share authenticated user information.
- Manage form inputs.
- Optimize application performance.

React Hooks make these tasks easier to organize and implement.

1.2 Why are Hooks important?

Consider this simple functional component:

JavaScript

```javascript
function Students() {

  return (
    <div>
      <h1>Student Management</h1>

      <button>Add Student</button>
    </div>
  );
}

export default Students;
```

This component displays a heading and a button.

Cont. ...

But what happens when a user clicks the button?

- Should a form appear?
- Should the student count increase?
- Should information be stored?
- Should the application communicate with the backend?

We need a way to manage these changes.

This is where React Hooks become useful.

For example, using `useState`, we can store and update information inside a functional component.

Cont. ...

For example, using `useState`, we can store and update information inside a functional component.

JavaScript

```
import { useState } from "react";

function Students() {

  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Student Management</h1>

      <p>Total Students: {count}</p>

      <button onClick={() => setCount(count + 1)}>
        Add Student
      </button>
    </div>
  );
}

export default Students;
```

Now, when the user clicks the button, the number increases.This is one of the simplest examples of React Hooks.

Why we need Hooks?

2. Why do we need React Hooks?

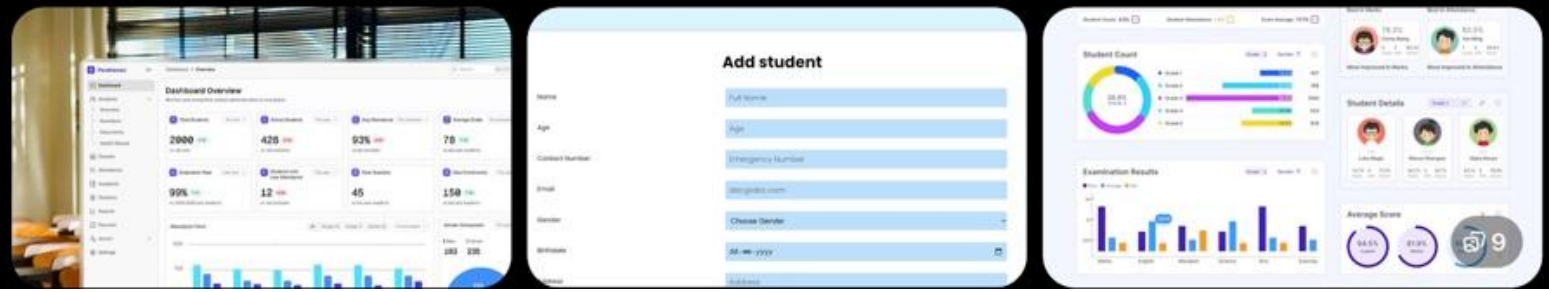
Let's consider a real-world university management application.

Suppose an administrator opens the Students page.

The application needs to:

1. Retrieve students from MongoDB through an Express API.
2. Store the retrieved student records.
3. Display the student information.
4. Show a loading message while retrieving data.
5. Display an error if the API request fails.
6. Update the student list when a new student is created.

Different Hooks help us implement these functionalities.



React Hooks Workflow

Hooks help functional components to manage state, side effects and more!

1 Functional Component

A React function component (e.g. Students.jsx)

Students.jsx

```
function Students() {
  // Hooks inside...
}
```

2 React Hooks in Action

- **useState** — Store data
- **useEffect** — Side effects
- **useContext** — Shared data
- **useRef** — DOM references

3 React Renders the Component

React processes the Hooks and updates the UI when required.

4 Updated User Interface

Displays the latest student information.

Student List

- Yitayew Solomon
- Sara Ahmed
- Daniel Tesfaye

5 The Big Picture

Hooks make your functional components powerful and easy to manage!

- Manage State
- Handle Side Effects
- Share Data
- Access DOM

In short: React Hooks work inside functional components to manage data, handle side effects, and keep the UI up to date.

Simple. Powerful. React!

Why we need State?

4.2 Why do we need useState?

Imagine a student registration form.

A user enters:

- First name.
- Last name.
- Email.
- Department.

The application needs to remember the entered information while the user interacts with the form.

We can use `useState` to store these values.

4.3 Syntax of useState

```
JavaScript
const [state, setState] = useState(initialValue);
```

Let's understand each part.

Part	Meaning
`useState`	React Hook
`state`	Current state value
`setState`	Function used to update the state
`initialValue`	Initial value of the state

For example:

```
JavaScript
const [count, setCount] = useState(0);
```

Example:

Now let's implement something more practical.

AddStudent.jsx

JavaScript

```
import { useState } from "react";

function AddStudent() {

  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  function handleSubmit(event) {

    event.preventDefault();

    console.log({
      name,
      email
    });

  }

  return (
    <div>

      <h1>Add Student</h1>

      <form onSubmit={handleSubmit}>

        <input
          type="text"
          placeholder="Student Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />

        <input
          type="email"
          placeholder="Student Email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />

        <button type="submit">
          Save Student
        </button>

      </form>

    </div>
  );
}

export default AddStudent;
```

Cont. ...

Let's understand the important properties.

value

```JavaScript
value={name}
```

This connects the input field to the React state.

onChange

```JavaScript
onChange={(e) => setName(e.target.value)}
```

This updates the state whenever the user enters or changes text.

event.preventDefault()

```JavaScript
event.preventDefault();
```

This prevents the browser from performing its normal form submission and reloading the page.

Managing multiple fields using one state object

Instead of creating separate state variables for every field, we can use one object.

JavaScript

```
import { useState } from "react";

function AddStudent() {

  const [student, setStudent] = useState({
    firstName: "",
    lastName: "",
    email: "",
    department: ""
  });

  function handleChange(event) {

    const { name, value } = event.target;

    setStudent(previous => ({
      ...previous,
      [name]: value
    }));

  }

  function handleSubmit(event) {

    event.preventDefault();

    console.log(student);

  }

  return (
    <form onSubmit={handleSubmit}>

      <input
        name="firstName"
        placeholder="First Name"
        value={student.firstName}
        onChange={handleChange}
      />

      <input
        name="lastName"
        placeholder="Last Name"
        value={student.lastName}
        onChange={handleChange}
      />

      <input
        name="email"
        placeholder="Email"
        value={student.email}
        onChange={handleChange}
      />

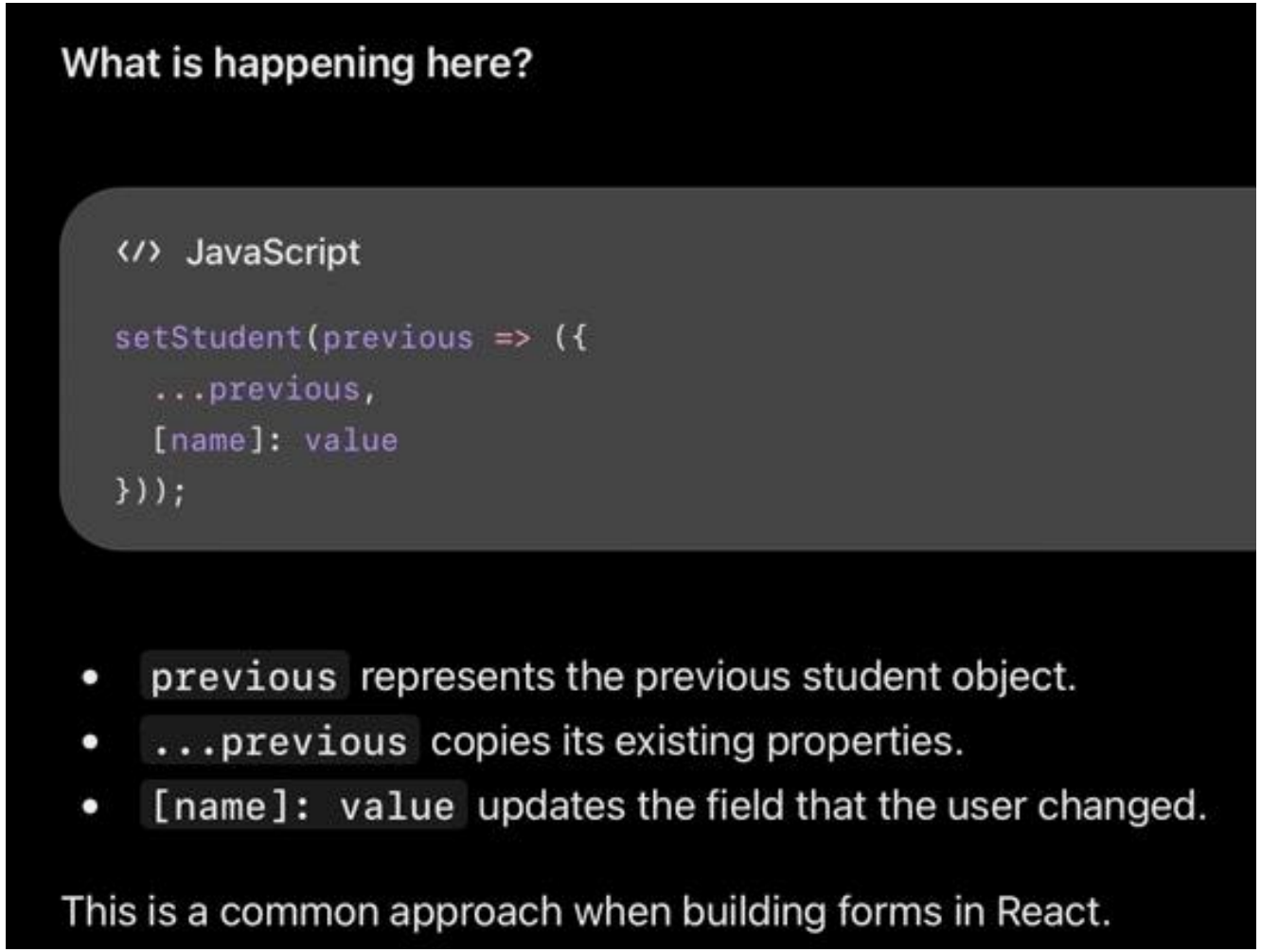
      <input
        name="department"
        placeholder="Department"
        value={student.department}
        onChange={handleChange}
      />

      <button type="submit">
        Save Student
      </button>

    </form>
  );
}

export default AddStudent;
```

Cont. ...


What is happening here?
JavaScript
setStudent(previous => ({
...previous,
[name]: value
}));
previous represents the previous student object.
...previous copies its existing properties.
[name]: value updates the field that the user changed.
This is a common approach when building forms in React.

useEffect Hook

5. useEffect Hook

Beginner to intermediate

5.1 What is useEffect?

`useEffect` is a React Hook used to synchronize a component with external systems.

Examples of external systems include:

- Backend APIs.
- Browser event listeners.
- Timers.
- WebSocket connections.
- Third-party libraries.
- Subscriptions.

In simple terms, `useEffect` allows a component to perform side-effect operations.

A side effect is an operation that interacts with something outside the normal rendering calculation of a component.

Why do we need useEffect?

5.2 Why do we need useEffect?

Imagine that an administrator opens the Students page.

The application needs to retrieve student records from the backend.

We need to:

1. Send an HTTP request.
2. Receive the response.
3. Store the student information.
4. Display the records.

We generally do not want to send that API request directly during every component render.

We can use `useEffect` to synchronize the component with the API.

Cont. ...

5.3 Syntax of useEffect

```javascript
useEffect(() => {

  // Side effect logic

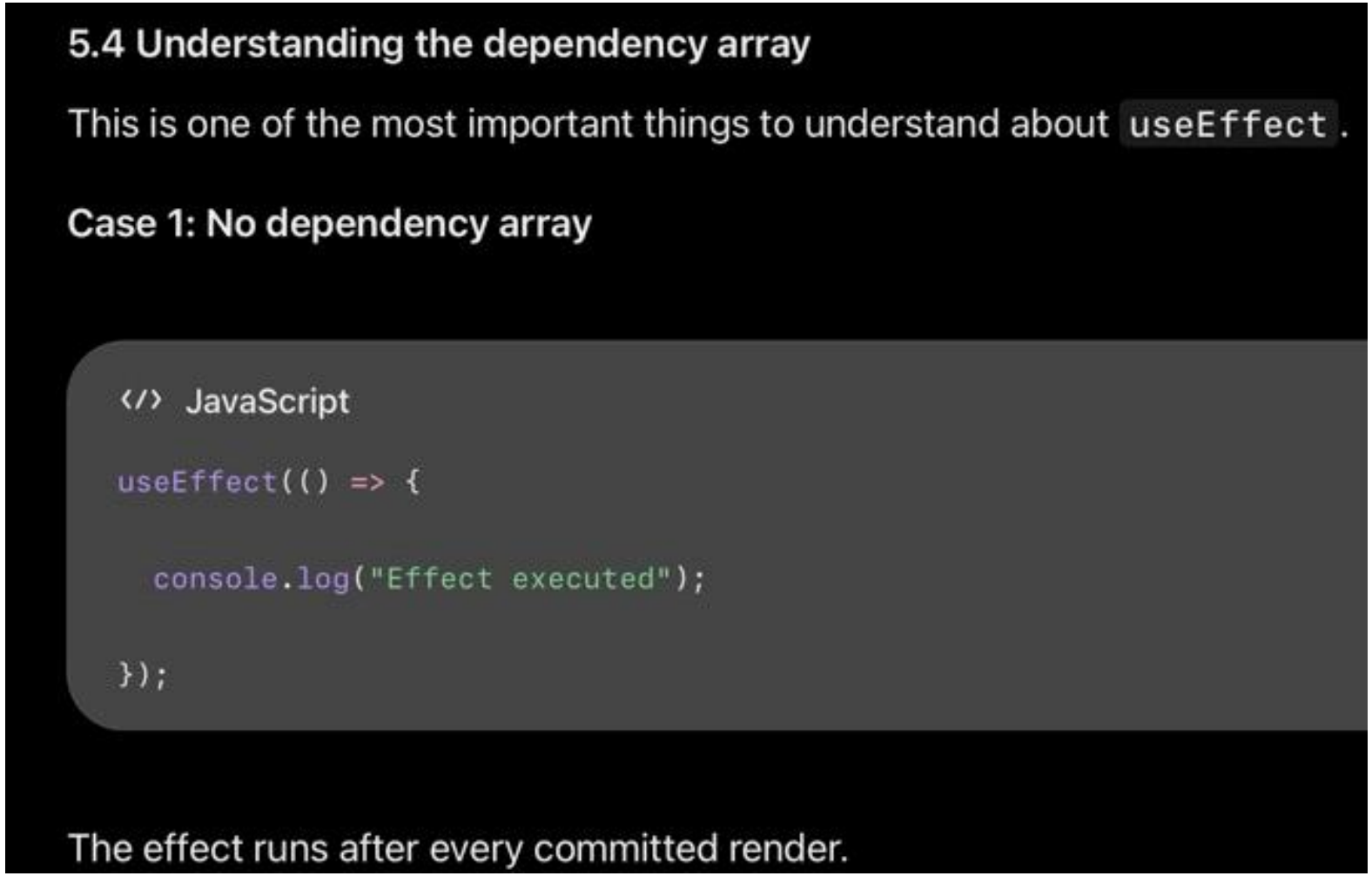
  return () => {
    // Optional cleanup
  };

}, [dependencies]);
```

There are three important parts:

Part	Meaning
Effect function	Contains the side-effect logic
Cleanup function	Cleans up resources when necessary
Dependency array	Determines when React should re-synchronize the effect

Cont. ...


5.4 Understanding the dependency array
This is one of the most important things to understand about useEffect.
Case 1: No dependency array
JavaScript
useEffect(() => {
console.log("Effect executed");
});
The effect runs after every committed render.

Cont. ...

Case 2: Empty dependency array

JavaScript

```
useEffect(() => {

  console.log("Effect executed");

}, []);
```

The effect runs when the component mounts.

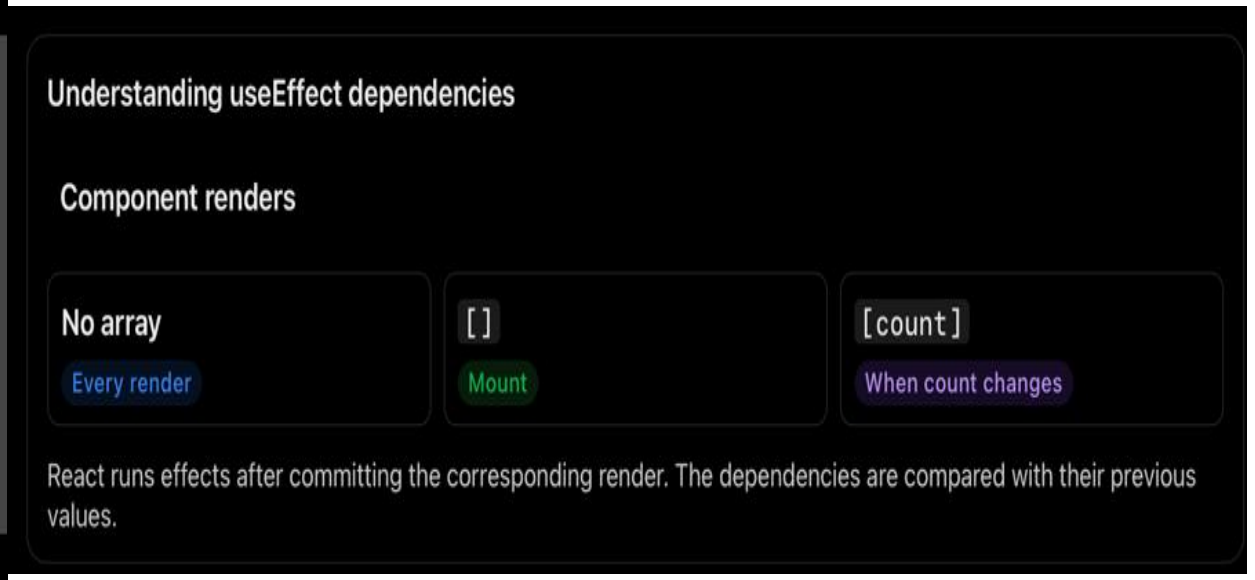
In development Strict Mode, React may perform an additional setup and cleanup cycle to help identify unsafe effects. Therefore, do not assume that an effect always executes exactly once in every environment.

Cont. ...

Case 3: Dependency array with a value

```javascript
useEffect(() => {
  console.log("Count changed");
}, [count]);
```

The effect re-synchronizes when `count` changes.



useContext Hook

6. useContext Hook

Intermediate level

6.1 What is useContext?

`useContext` is a React Hook that allows components to access shared information without passing data manually through multiple levels of components.

To understand this, we first need to understand a concept called *prop drilling*.

6.2 What is prop drilling?

Prop drilling happens when data is passed from a parent component to a deeply nested child component through several intermediate components.

Cont. ...

6.3 Why do we need useContext?

In a large application, passing information through many component levels becomes difficult to maintain.

For example, your MERN application may need to share:

- Authenticated user information.
- User roles.
- User permissions.
- Application theme.
- Language preferences.
- Notifications.

Instead of passing these values through multiple components, we can use React Context.

How to use useContext

There are three main steps:

1. Create a Context.
2. Provide the shared value.
3. Consume the value using `useContext`.

Step 1: Create AuthContext.jsx

JavaScript

```
import { createContext } from "react";

export const AuthContext = createContext(null);
```

`createContext()` creates a context object that can hold shared information.

Cont. ...

Step 2: Create AuthProvider.jsx

JavaScript

```
import { useState } from "react";

import { AuthContext } from "./AuthContext";

function AuthProvider({ children }) {

  const [user, setUser] = useState({
    name: "Abebe Kebede",
    role: "Administrator"
  });

  return (
    <AuthContext.Provider value={{ user, setUser }}>

      {children}

    </AuthContext.Provider>
  );
}

export default AuthProvider;
```

Here:

- `AuthContext.Provider` makes the information available to descendant components.
- `value` contains the information we want to share.
- `children` represents the components inside the provider.

Cont. ...

Step 3: Wrap the application

In your `main.jsx`:

JavaScript

```
import React from "react";
import ReactDOM from "react-dom/client";

import App from "./App";
import AuthProvider from "./context/AuthProvider";

ReactDOM.createRoot(
  document.getElementById("root")
).render(

  <React.StrictMode>

    <AuthProvider>
      <App />
    </AuthProvider>

  </React.StrictMode>

);
```

Step 4: Consume the Context

JavaScript

```
import { useContext } from "react";

import { AuthContext } from "../context/AuthContext";

function UserProfile() {

  const { user } = useContext(AuthContext);

  return (
    <div>

      <h2>Welcome, {user.name}</h2>

      <p>Role: {user.role}</p>

    </div>
  );
}

export default UserProfile;
```

useRef Hook

1. useRef Hook

Beginner to Intermediate

What is useRef?

`useRef` is a React Hook that allows us to store a value between renders without causing the component to render again when that value changes.

It is also used to directly access HTML elements.

Why do we need it?

For example, when registering a student, we may want the cursor to automatically appear inside the Student Name input field.

Cont. ...

Example: Automatically focus an input field

JavaScript

```
import { useRef } from "react";

function StudentForm() {

  const nameRef = useRef(null);

  function focusInput() {
    nameRef.current.focus();
  }

  return (
    <div>
      <h2>Student Registration</h2>

      <input
        ref={nameRef}
        placeholder="Enter student name"
      />
      <button onClick={focusInput}>
        Focus Input
      </button>
    </div>
  );
}

export default StudentForm;
```

How does it work?

- `useRef(null)` creates a reference.
- `ref={nameRef}` connects the reference to the input.
- `nameRef.current` accesses the input element.
- `.focus()` places the cursor inside the input.

Real-world examples:

- Automatically focusing input fields.
- Accessing video and audio elements.
- Managing timers.
- Storing values that do not need to update the interface.

Remember: `useState` updates the interface when its value changes, whereas updating `useRef` does not trigger a re-render.

useReducer Hook

What is useReducer?

`useReducer` is a React Hook used to manage state, especially when an application has multiple related state changes.

It is an alternative to `useState`.

Why do we need it?

Imagine an enrollment management system where an administrator can:

- Approve enrollment.
- Reject enrollment.
- Cancel enrollment.

Instead of managing many related updates separately, we can use `useReducer` to organize them.

Cont. ...

Example: Enrollment status management

JavaScript

```
import { useReducer } from "react";

function reducer(state, action) {

  switch (action.type) {

    case "APPROVE":
      return { status: "Approved" };

    case "REJECT":
      return { status: "Rejected" };

    case "CANCEL":
      return { status: "Cancelled" };

    default:
      return state;
  }
}
```

JavaScript

```
function Enrollment() {

  const [state, dispatch] = useReducer(
    reducer,
    { status: "Pending" }
  );

  return (
    <div>

      <h2>Status: {state.status}</h2>

      <button onClick={() =>
        dispatch({ type: "APPROVE" })}>
        Approve
      </button>

      <button onClick={() =>
        dispatch({ type: "REJECT" })}>
        Reject
      </button>
```

JavaScript

```
      <button onClick={() =>
        dispatch({ type: "CANCEL" })}>
        Cancel
      </button>

    </div>
  );
}

export default Enrollment;
```

Cont. ...

How does it work?

1. `state` stores the current enrollment status.
2. `dispatch()` sends an action.
3. `reducer()` checks the action type.
4. The reducer returns the new state.
5. React updates the interface.

Remember: `useReducer` is useful when state management involves multiple related actions.

useMemo Hook

What is useMemo?

`useMemo` is a React Hook that remembers the result of a calculation so that React does not need to repeat the same calculation unnecessarily.

Why do we need it?

Imagine a university has thousands of student records.

When users search for a student, React needs to filter the records.

If the component renders again for an unrelated reason, we may want to avoid repeating an expensive filtering calculation.

Cont. ...

Example: Filtering students

JavaScript

```
import { useMemo, useState } from "react";

const students = [
  { id: 1, name: "Abebe" },
  { id: 2, name: "Selam" },
  { id: 3, name: "Hana" }
];

function Students() {

  const [search, setSearch] = useState("");

  const filteredStudents = useMemo(() => {

    return students.filter(student =>
      student.name.toLowerCase().includes(
        search.toLowerCase()
      )
    );

  }, [search]);
```

JavaScript

```
  return (
    <div>

      <h2>Student List</h2>

      <input
        placeholder="Search student"
        value={search}
        onChange={e => setSearch(e.target.value)}
      />

      {filteredStudents.map(student => (
        <p key={student.id}>
          {student.name}
        </p>
      ))}

    </div>
  );
}

export default Students;
```

Cont. ...

How does it work?

- `useMemo` remembers the filtered student list.
- When `search` changes, React performs the filtering again.
- When `search` does not change, React can reuse the previous result.

Real-world examples:

- Filtering student records.
- Sorting course lists.
- Calculating dashboard statistics.
- Processing large datasets.

Remember: `useMemo` remembers a calculated value. It is a performance optimization, not something required for every calculation.

useCallback Hook

What is useCallback?

`useCallback` is a React Hook that remembers a function between renders.

Why do we need it?

In large React applications, components frequently pass functions to other components.

Sometimes, creating a new function on every render can cause unnecessary child component rendering.

`useCallback` helps preserve the function reference.

Cont. ...

Example: Delete student

JavaScript

```
import { useState, useCallback } from "react";

function Students() {

  const [students, setStudents] = useState([
    { id: 1, name: "Abebe" },
    { id: 2, name: "Selam" }
  ]);

  const deleteStudent = useCallback((id) => {

    setStudents(previous =>
      previous.filter(student => student.id !== id)
    );

  }, []);

  return (
    <div>

      <h2>Students</h2>
```

JavaScript

```
      {students.map(student => (

        <div key={student.id}>

          {student.name}

          <button onClick={() =>
            deleteStudent(student.id)}>
            Delete
          </button>

        </div>

      ))}

    </div>
  );
}

export default Students;
```

Cont. ...

How does it work?

- `useCallback` preserves the function reference.
- The function deletes a student using the student's ID.
- The state updates and React displays the updated student list.

Remember: `useCallback` remembers a function, while `useMemo` remembers a calculated value.

Cont. ...

What is useId?

`useId` is a React Hook used to generate unique IDs for HTML elements, especially form inputs and their labels.

Why do we need it?

In a student registration form, each input field should have a unique ID so that its label is properly connected to it.

Cont. ...

Example

JavaScript

```
import { useId } from "react";

function StudentForm() {

  const emailId = useId();

  return (
    <div>

      <label htmlFor={emailId}>
        Student Email
      </label>

      <input
        id={emailId}
        type="email"
        placeholder="Enter email"
      />
              }

    </div>
              export default StudentForm;
  );
```

How does it work?

- `useId()` generates a unique ID.
- `htmlFor` connects the label to the input.
- `id` identifies the input field.

Real-world examples:

- Student registration forms.
- Login forms.
- Reusable form components.
- Accessible user interfaces.

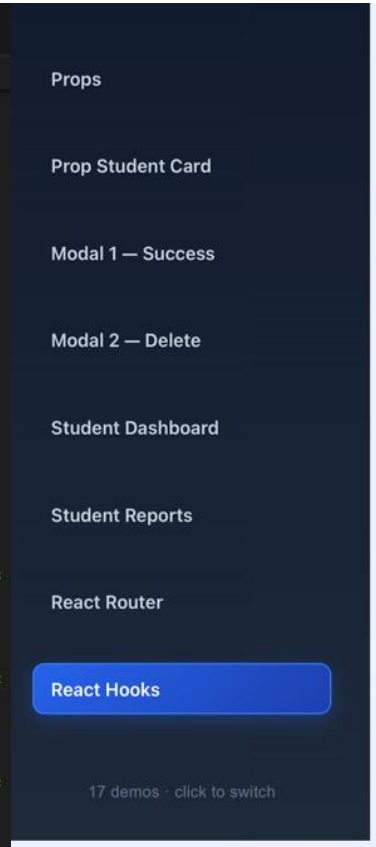
Remember: `useId` helps connect HTML elements. It should not be used to generate database IDs or list keys.

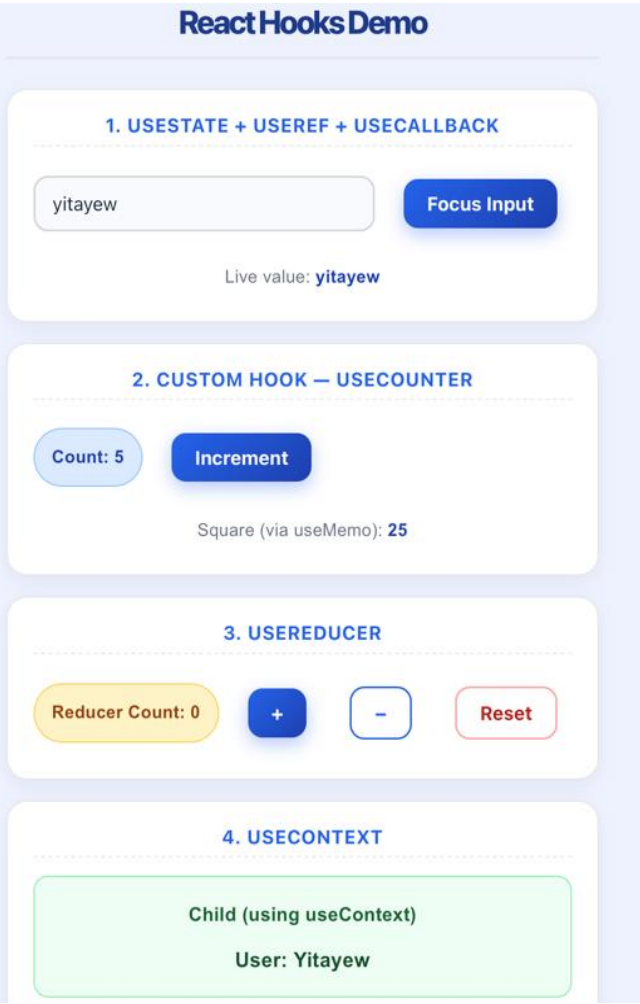
Example

App.jsx M Hook.jsx U

React_App > src > Component > Hook.jsx > ...

```
23  import React, {
24    createContext,
25    useState,
26    useEffect,
27    useContext,
28    useRef,
29    useMemo,
30    useCallback,
31    useReducer,
32  } from "react";
33
34  import "./Hook.css";
35
36  // ============================================================
37  // CONTEXT — shared value available to any descendant without
38  // prop-drilling. Here it stores the current user's name.
39  // ============================================================
40  const UserContext = createContext();
41
42  // ============================================================
43  // CUSTOM HOOK — useCounter
44  //
45  // A reusable piece of stateful logic. Its name starts with
46  // "use" (React rule for custom hooks). Any component can call
47  // it and get its own independent counter + increment function.
48  // ============================================================
```





useTransition Hook

What is useTransition?

`useTransition` is a React Hook that allows React to treat certain state updates as less urgent.

Why do we need it?

Imagine that a student management system contains thousands of records.

When users type in a search box, the input should respond immediately, even if updating the student list requires more processing.

`useTransition` helps prioritize the important user interaction.

Cont. ...

Example

JavaScript

```
import {
  useState,
  useTransition
} from "react";

function StudentSearch() {

  const [search, setSearch] = useState("");

  const [filter, setFilter] = useState("");

  const [isPending, startTransition] =
    useTransition();

  function handleSearch(e) {

    const value = e.target.value;

    setSearch(value);

    startTransition(() => {
      setFilter(value);
    });

  }

  return (
    <div>

      <h2>Student Search</h2>

      <input
        value={search}
        onChange={handleSearch}
        placeholder="Search students"
      />

      {isPending && <p>Updating results...</p>}

      <p>Search result for: {filter}</p>

      </div>
    );
  }

export default StudentSearch;
```

Cont. ...

How does it work?

- `search` updates immediately.
- `startTransition()` marks the filter update as non-urgent.
- `isPending` indicates whether the transition is still pending.

Remember: `useTransition` helps maintain a responsive user interface when performing expensive UI updates.

useDeferredValue Hook

What is useDeferredValue?

`useDeferredValue` is a React Hook that allows React to delay updating a value when more urgent updates need priority.

Why do we need it?

Imagine that a search input is connected to a large student list.

We want the user to type immediately without waiting for the entire student list to update.

Cont. ...

Example

JavaScript

```
import {
  useState,
  useDeferredValue,
  useMemo
} from "react";

const students = [
  "Abebe",
  "Selam",
  "Hana",
  "Dawit"
];

function StudentSearch() {

  const [search, setSearch] = useState("");

  const deferredSearch =
    useDeferredValue(search);
```

JavaScript

```
  const filteredStudents = useMemo(() => {

    return students.filter(student =>
      student.toLowerCase().includes(
        deferredSearch.toLowerCase()
      )
    );

  }, [deferredSearch]);

  return (
    <div>

      <input
        value={search}
        onChange={e => setSearch(e.target.value)}
        placeholder="Search student"
      />

      {filteredStudents.map(student => (
        <p key={student}>{student}</p>
      ))}
```

```
    </div>
  );
}

export default StudentSearch;
```

Cont. ...

How does it work?

1. The user enters a search keyword.
2. The input updates immediately.
3. React may temporarily retain the previous deferred value.
4. The filtered results update when React can process them.

Remember: `useDeferredValue` is useful when displaying large or expensive search results.

Custom Hooks

What is a Custom Hook?

A Custom Hook is a reusable JavaScript function that contains React Hook logic.

It allows us to share the same functionality across multiple components.

Why do we need it?

Imagine that Students, Courses, and Enrollments pages all need to retrieve data from the backend.

Instead of writing the same API request logic repeatedly, we can create reusable Custom Hooks.

Cont. ...

Example: Creating a Custom Hook

useStudents.js

JavaScript

```
import { useState, useEffect } from "react";

function useStudents() {

  const [students, setStudents] = useState([]);

  const [loading, setLoading] = useState(true);

  useEffect(() => {

    async function fetchStudents() {

      try {

        const response = await fetch(
          "http://localhost:5000/api/students"
        );

        const result = await response.json();

        setStudents(result.data);

      } catch (error) {

        console.log(error);

      } finally {

        setLoading(false);

      }

    }

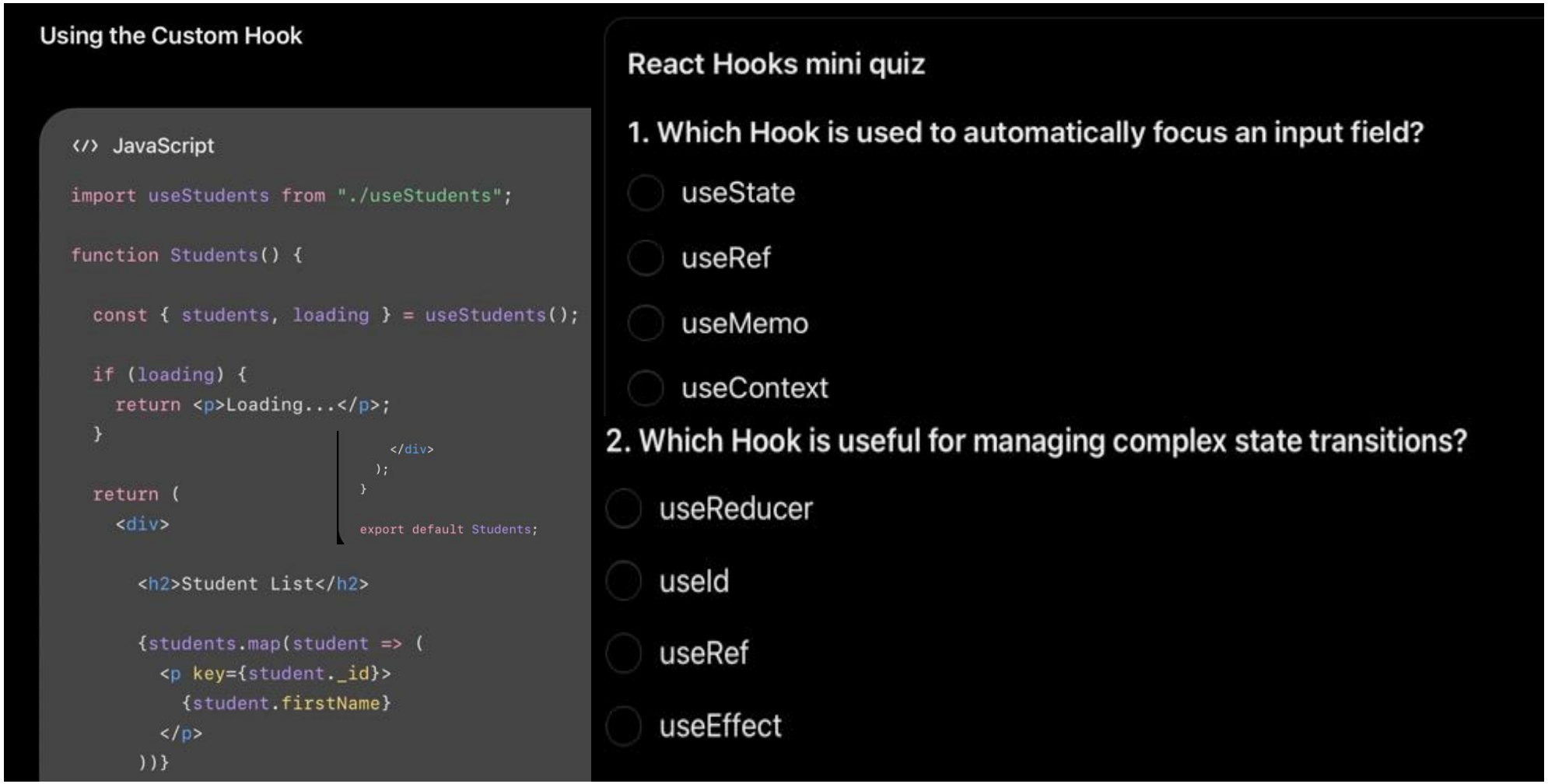
    fetchStudents();

  }, []);

  return { students, loading };
}

export default useStudents;
```

Cont. ...


Using the Custom Hook
JavaScript
import useStudents from "./useStudents";
function Students() {
const { students, loading } = useStudents();
if (loading) {
return Loading... ;
}
return (

<h2>Student List</h2>
{students.map(student => (

{student.firstName}

))}

);
}
export default Students;
React Hooks mini quiz
1. Which Hook is used to automatically focus an input field?
useState
useRef
useMemo
useContext
2. Which Hook is useful for managing complex state transitions?
useReducer
useId
useRef
useEffect

Cont. ...

3. Which Hook remembers the result of a calculation?

- useCallback
- useContext
- useMemo
- useRef

4. Which Hook remembers a function reference?

- useMemo
- useCallback
- useState
- useId

5. Which Hook generates IDs for accessible form elements?

- useReducer
- useEffect
- useId
- useTransition

6. What is the main purpose of Custom Hooks?

- Create database tables
- Reuse React logic
- Replace React Router
- Style components


Thank you
GRATEFUL